2.3 Encapsulation, Accessors, and Property Hooks

Chapter 18 protects properties with traditional accessor and mutator methods. PHP 8.4 adds property hooks, a modern way to keep property syntax while controlling reads and writes. Learn the traditional form first because it is common in existing PHP.

Getters and setters

An **accessor** (getter) returns a value. A **mutator** (setter) changes a value. They let the object screen data before it enters its state.

Update `src/Tutorial/Class2/Person.php`:

```php
class Person
{
    private string $displayName;

    public function getDisplayName(): string
    {
        return $this->displayName;
    }

    public function setDisplayName(string $displayName): void
    {
        $displayName = trim($displayName);

        if ($displayName === '') {
            throw new InvalidArgumentException('Name cannot be empty.');
        }

        $this->displayName = $displayName;
    }
}
```

The setter establishes an invariant: a valid `Person` never has an empty display name. Throwing an exception makes invalid input visible; silently ignoring it leaves calling code unsure whether the update worked.

Not every property needs both methods. Omit a setter when outside code should not replace a value, and omit a getter when the stored representation is an implementation detail (e.g., a private property used for internal calculations).

PHP 8.4 property hooks

A property hook intercepts a read (`get`) or write (`set`) while callers use ordinary property syntax. This removes boilerplate without giving up validation.

Let's update our `Person` class to use property hooks:

```php
class Person
{
    public string $displayName {
        set(string $value) {
            $value = trim($value);

            if ($value === '') {
                throw new InvalidArgumentException('Name cannot be empty.');
            }

            $this->displayName = $value;
        }
    }

    public string $birthDate {
        set(string $value) {
            $ts = strtotime(trim($value));

            if ($ts === false) {
                throw new InvalidArgumentException('Invalid date format.');
            } elseif ($ts > time()) {
                throw new InvalidArgumentException('Birth date cannot be in the
future.');
            }

            $this->birthDate = $value;
        }
    }
}

$student = new Person();
$student->displayName = '  Joe  ';
$student->birthDate = '1990-01-01';

echo $student->displayName; // Joe
```

These are **backed properties** because their hooks read or write the property itself. PHP recognizes that access inside the hook refers to backing storage rather than recursively calling the hook.

Virtual properties

A hook can compute a value that is not stored:

```php
class Person
{
    public float $age {
        get => (new DateTime())->diff(new DateTime($this->birthDate))->y;
    }
}
```

`age` is virtual: `$person->age` calculates from a stored property. Because it has no `set` hook, outside code cannot assign to it.

Asymmetric property visibility

Hooks control **what happens** during access. Asymmetric visibility controls **who may write**. PHP 8.4 allows a property to be publicly readable but more narrowly writable. For example, update `src/Tutorial/Class2/Registration.php`:

```php
class Registration
{
    private Person $student;
    private Course $course;
    private string $registrationTime;
    public private(set) string $status = 'registered';

    public function drop(): void
    {
        $this->status = 'dropped';
    }

    public function getStudent(): Person
    {
        return $this->student;
    }

    public function setStudent(Person $student): void
    {
        $this->student = $student;
    }

    public function getCourse(): Course
    {
        return $this->course;
    }

    public function setCourse(Course $course): void
    {
        $this->course = $course;
    }

    public function getRegistrationTime(): string
    {
        return $this->registrationTime;
    }

    public function setRegistrationTime(string $registrationTime): void
    {
        $this->registrationTime = $registrationTime;
    }
}
```

`$status` is publicly readable but can only be written inside the class. The `drop()` method changes the status, but outside code cannot assign to it directly.

Use asymmetric visibility when reading may be public but writing must stay inside the class or inheritance hierarchy.

Property hooks cannot be combined with `readonly` properties. Property hooks apply to non-static properties in PHP 8.4.

For the complete language rules and additional examples, see the PHP manual pages for property hooks and asymmetric property visibility.